

1. Create a Generic Data Model

```
public class GenericFormData {
    private Map<String, String> data;

    public GenericFormData() {
        this.data = new HashMap<>();
    }

    public GenericFormData(Map<String, String> data) {
        this.data = data != null ? data : new HashMap<>();
    }

    // Getters and setters
    public Map<String, String> getData() {
        return data;
    }

    public void setData(Map<String, String> data) {
        this.data = data;
    }

    // Convenience methods
    public String getValue(String key) {
        return data.get(key);
    }

    public void setValue(String key, String value) {
        data.put(key, value);
    }
}
```

2. Custom Validator Implementation

```
@Component
public class GenericFormValidator implements Validator {
    private final Map<String, List<ValidationRule>> validationRules = new HashMap<>();

    @PostConstruct
    public void initializeValidations() {
        // Configure validations by field name
        validationRules.put("test-textarea-1", Arrays.asList(
            new RequiredValidationRule("Test textarea 1 is required"),
            new LengthValidationRule(10, 500, "Must be between 10-500 characters")
        ));

        validationRules.put("test-textinput-4", Arrays.asList(
            new RequiredValidationRule("Test input 4 is required"),
            new PatternValidationRule("^[A-Za-z\\s]+$", "Only letters and spaces allowed")
        ));

        validationRules.put("textArea_test", Arrays.asList(
            new LengthValidationRule(0, 1000, "Cannot exceed 1000 characters")
        ));
    }

    @Override
    public boolean supports(Class<?> clazz) {
        return GenericFormData.class.equals(clazz);
    }

    @Override
    public void validate(Object target, Errors errors) {
        GenericFormData formData = (GenericFormData) target;
        Map<String, String> data = formData.getData();

        // Validate each field in the data map
        for (Map.Entry<String, String> entry : data.entrySet()) {
            String fieldName = entry.getKey();
            String value = entry.getValue();

            List<ValidationRule> rules = validationRules.get(fieldName);
            if (rules != null) {
                for (ValidationRule rule : rules) {

```

```

        if (!rule.isValid(value)) {
            errors.rejectValue("data[" + fieldName + "]",
                               rule.getErrorCode(),
                               rule.getErrorMessage());
        }
    }
}

// Check for required fields that might be missing
for (Map.Entry<String, List<ValidationRule>> entry : validationRules.entrySet()) {
    String fieldName = entry.getKey();
    if (!data.containsKey(fieldName) || data.get(fieldName) == null) {
        // Check if any rule is a required rule
        boolean hasRequiredRule = entry.getValue().stream()
            .anyMatch(rule -> rule instanceof RequiredValidationRule);

        if (hasRequiredRule) {
            errors.rejectValue("data[" + fieldName + "]",
                               "required",
                               fieldName + " is required");
        }
    }
}
}
}
}
}

```

3. Validation Rule Interfaces and Implementation

```

public interface ValidationRule {
    boolean isValid(String value);
    String getErrorMessage();
    String getErrorCode();
}

public class RequiredValidationRule implements ValidationRule {
    private final String errorMessage;

    public RequiredValidationRule(String errorMessage) {
        this.errorMessage = errorMessage;
    }

    @Override
    public boolean isValid(String value) {
        return value != null && !value.trim().isEmpty();
    }

    @Override
    public String getErrorMessage() {
        return errorMessage;
    }

    @Override
    public String getErrorCode() {
        return "required";
    }
}

public class LengthValidationRule implements ValidationRule {
    private final int minLength;
    private final int maxLength;
    private final String errorMessage;

    public LengthValidationRule(int minLength, int maxLength, String errorMessage) {
        this.minLength = minLength;
        this.maxLength = maxLength;
        this.errorMessage = errorMessage;
    }

    @Override
    public boolean isValid(String value) {
        if (value == null) return true; // Let required rule handle null values
        return value.length() >= minLength && value.length() <= maxLength;
    }

    @Override
    public String getErrorMessage() {
        return errorMessage;
    }

    @Override

```



```

        Model model) {
    // Manual validation using custom validator
    genericFormValidator.validate(formData, bindingResult);

    if (bindingResult.hasErrors()) {
        model.addAttribute("formData", formData);
        return "test"; // Return to form with errors
    }
    // Save data if validation passes
    saveData(formData.getData());
    return "redirect:/success";
}
// Alternative approach using @Valid annotation
@PostMapping("/test-auto")
public String submitTestAuto(@Valid @ModelAttribute("formData") GenericFormData formData,
                             BindingResult bindingResult,
                             Model model) {
    if (bindingResult.hasErrors()) {
        model.addAttribute("formData", formData);
        return "test";
    }
    saveData(formData.getData());
    return "redirect:/success";
}
private void saveData(Map<String, String> data) {
    // Your save logic here
    System.out.println("Saving data: " + data);
}
}

```

5. Configuration for Automatic Validation

```

@Configuration
public class ValidationConfig implements WebMvcConfigurer {
    @Autowired
    private GenericFormValidator genericFormValidator;

    @Override
    public Validator getValidator() {
        return genericFormValidator;
    }
}

```

6. Thymeleaf Template Updates

Test Textarea 1

Test Input 4

7. Enhanced Version with Configuration-Based Rules

You can also make it even more flexible by loading validation rules from configuration:

```

@Component
@ConfigurationProperties(prefix = "form.validation")
public class ValidationRuleConfig {
    private Map<String, FieldValidationConfig> fields = new HashMap<>();

    public static class FieldValidationConfig {
        private boolean required;
        private Integer minLength;
        private Integer maxLength;
        private String pattern;
        private String requiredMessage;
        private String lengthMessage;
        private String patternMessage;

        // getters and setters
    }
    // getters and setters
}

```

Benefits of This Approach:

1. **Spring Integration:** Full integration with Spring's validation framework
2. **Automatic Binding:** Works with `@Valid` annotation
3. **Error Handling:** Seamless integration with `BindingResult`
4. **Flexibility:** Can handle any field names dynamically
5. **Reusability:** Generic model works across different forms
6. **Thymeleaf Integration:** Easy error display in templates

This combines the best of both worlds: Spring's validation framework with your flexible data map approach!